

Novelty Library WebApp

Novelty Library is a React/Vite web application for book discovery, reader reviews, community engagement, reader profiles and editorial administration.

This documentation set was generated from the uploaded NL_WA_1_35.zip archive after inspecting the NL_WA_1_75 application source.

Documentation

- [PRD.md](#) (./PRD.md) — product requirements, users, workflows, functional/non-functional requirements and success metrics.
- [ARCHITECTURE.md](#) (./ARCHITECTURE.md) — application structure, data flow, Supabase architecture, storage, RPCs and security boundaries.
- [DESIGN.md](#) (./DESIGN.md) — visual language, color tokens, typography, component behavior, profile-card design and responsive rules.
- [RULES.md](#) (./RULES.md) — engineering, product, security, data, UI and change-management invariants.
- [TASKS.md](#) (./TASKS.md) — implementation baseline, production verification backlog and future work.

Stack

- React 18
- TypeScript
- Vite
- Tailwind CSS
- Supabase
- Framer Motion
- Lucide React
- DOMPurify
- html-to-image
- Vite PWA

Core product areas

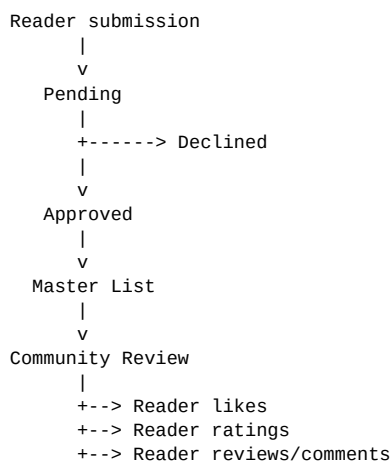
Public reading

- Home.
- Review shelf.
- Individual review pages.
- Analytics.
- About.
- Review guidelines.
- Legal/cookie pages.

Reader accounts

- Email/password authentication.
- Google OAuth.
- Password recovery.
- Reader profiles.
- Configurable profile questions.
- Public profiles.
- Profile-card generation/download.
- Stories.
- Following/followers.
- Notifications.

Review workflow



Important rating distinction

The platform has two different concepts:

R/W Rating — the original reviewer's rating/opinion.

NL Rating — the cumulative average of reader ratings submitted on the community review post.

The current implementation intentionally does not include the R/W Rating in the NL Rating calculation.

Local development

Install dependencies:

```
npm install
```

Start development server:

```
npm run dev
```

Type-check:

```
npm run typecheck
```

Lint:

```
npm run lint
```

Production build:

```
npm run build
```

Preview production build:

```
npm run preview
```

Environment variables

The frontend expects at minimum:

```
VITE_SUPABASE_URL=  
VITE_SUPABASE_ANON_KEY=
```

Never place a Supabase service-role key in the frontend environment.

Supabase requirements

The application expects a Supabase project containing the tables, storage buckets and RPC/database functions described in `ARCHITECTURE.md`.

The application uses **one Supabase client**. Do not add a second `createClient()` instance for a feature.

Important source locations

```
src/  
■■■ App.tsx  
■■■ components/  
■■■ lib/  
■   ■■■ auth.tsx  
■   ■■■ reviews.ts  
■   ■■■ engagement.ts  
■   ■■■ profileQuestions.ts  
■   ■■■ profileCoreFields.ts  
■   ■■■ profileLayout.ts  
■   ■■■ social.ts  
■   ■■■ moderation.ts  
■   ■■■ supabase.ts  
■■■ pages/  
■■■ types/
```

Main admin areas

Admin access is protected by the backend `is_admin()` function.

The current admin hub provides:

- Book Reviews.
- Admins & Contacts.
- Users & Accounts.
- Finances and Payments.
- WebApp Pages Moderation.

- Supabase Dashboard shortcut.

Profile configuration

Admins can configure profile questions from the WebApp Pages Moderation area. Supported question types include:

- Short answer.
- Paragraph.
- Number.
- Year.
- URL.
- Single select.
- Multiple select.
- Image upload.

Multi-select questions can have a maximum-selection cap and optional A–Z sorting. Profile-card appearance can be configured as a tag, question + answer, or answer-only.

Profile-card export

The current card supports:

Format	Export size
9:16	1080 × 1920
16:9	1600 × 900
3:4	1200 × 1600
4:5	1080 × 1350
1:1	1080 × 1080

The profile-card renderer adapts its layout by aspect ratio and progressively fits content when necessary.

Operational principles

1. Protect editorial trust.
2. Keep reviewer and community ratings separate.
3. Sanitize user-generated text.
4. Validate external URLs.
5. Enforce admin permissions server-side.
6. Keep non-critical integrations from breaking the reading experience.
7. Preserve light/dark accessibility and reduced-motion behavior.
8. Keep documentation synchronized with substantial product/architecture changes.

Deployment checklist

Before production deployment:

```
[ ] npm run typecheck
[ ] npm run lint
[ ] npm run build
[ ] Verify production environment variables
[ ] Verify Supabase OAuth redirects
[ ] Verify RLS and admin RPC authorization
[ ] Verify review submission/publish flow
[ ] Verify public review pages
[ ] Verify profile/public-profile privacy
[ ] Verify profile-card export
[ ] Verify community engagement
[ ] Verify Adsterra configuration
[ ] Verify mobile and dark theme
```

Documentation maintenance

When behavior changes:

- Product changes → update PRD.md.
- Architecture/data changes → update ARCHITECTURE.md.
- Visual/UI changes → update DESIGN.md.
- New invariant/security rule → update RULES.md.
- Implementation status → update TASKS.md.
- Setup/deployment changes → update README.md.

Version 1.40 profile update

- Restored the 24-hour story upload UI beside the profile card using the 1.36 layout and styling.
- Restored the follower/following privacy controls beside the profile card using the 1.36 presentation.
- Placed the current 1.39 profile-question buttons directly below the story/privacy area while keeping their current question editing and visibility controls unchanged.

Version 1.41 profile-admin update

- Built-in profile fields (Instagram, Reading Since, Books Read This Month, Total Books Read, Favourite Book/Author/Genre) are now managed as **section columns** in Admin -> WebApp Pages Moderation -> Profile Page, exactly like question sections.
- Each built-in field can be dragged to reorder it, dragged into the other built-in section (Reader identity <-> Reading journey), hidden/shown with the eye button, or edited (label, placeholder, section, default visibility). Name and Email stay locked at the top of the identity section.
- The reader profile page renders built-in fields by their effective section; saving either built-in section persists all built-in field values, so a moved field never loses data.
- Section-tile drag fixes: tiles no longer contain a full-width <button> (Firefox cannot start a drag from inside one), drops between tiles now save, and the new order shows immediately and reloads from the server if the save fails.

Version 1.42 - Reading Journey becomes a normal section

- Admin -> WebApp Pages Moderation -> Profile Page shows a **Convert to a section** button for the built-in Reading Journey. It creates a normal question section named **Reading Journey** (same header, text, position and visibility as the built-in one) and six ordinary questions with the same names: Reading Since, Books Read This Month, Total Books Read, Favourite Book, Favourite Author, Favourite Genre. Admin edits already made to those built-in fields (label, placeholder, default visibility, hidden, order) are carried over. The action is idempotent.

- After conversion they are edited, dragged, moved between sections, hidden and deleted exactly like any other question. Their keys equal the old column names, so readers' existing answers are kept: the values still live in the `profiles` columns and are read/written by the question fields. The profile card, public profile, onboarding and admin user view are unchanged.
- The built-in Reading Journey tile is retired after conversion. **Identity** keeps its name and fields (Name, Email, Instagram, social links).
- The "Published with Novelty Library" count card moved from Reading Journey into **Identity**.
- Cards no longer render converted reading fields twice, and a converted question that is hidden or off-card switches the matching card row off, on both the reader profile and the public profile.

Version 1.60 - Reviewer verdict

- New **compulsory** submit-form question, placed right before the review text and ratings: **Perfection**, **Go for it** or **Timepass** (stored as `perfection`, `go_for_it`, `timepass`).
- The verdict flows reviews -> `master_list` (Accepted) -> `community_reviews` (Published). It is only written when set, so legacy rows, Google Sheets imports and admin queue transitions are unaffected (NULL = no banner).
- Published review pages and feed cards show an teal/cyan gradient **verdict tag** (small violet accent) with a unique SVG symbol per verdict.
- On submit/save, a soft **verdict vs R/W rating check** asks the reviewer to confirm mismatches (Perfection < 8.5, Go for it < 5.5, Timepass > 7.5). Thresholds live in `src/lib/verdict.ts` (VERDICT_RATING_RULES).
- Database: run `sql/verdict_column.sql` (idempotent; the live project already has the column and `*_verdict_chk` CHECK constraints).
- Admin -> Accepted Reviews: the **Verdict** can be set for older reviews (Cards -> edit, or click the Verdict cell in the table). Saving also copies just the verdict onto the already-published copy, so the banner appears without a republish. A **No verdict only** filter lists reviews still missing one, and an **exact Review No.** box works like the Publishing Queue's.

Version 1.75 — Profile card and reader Q&A

- Added a distinct average R/W Rating star tile beside the average NL Rating tile. The two ratings remain separate.
- Restyled profile-card question/answer boxes with clearer label hierarchy, stronger cyan/teal treatment, and improved wrapping.
- Removed reader controls for choosing card questions and choosing Tag versus Answer; Tag/Answer presentation is admin-managed.
- Automatically limits Advanced Reader (Reading Identity) card content to the first two questions. Questions in that section after the first two and later sections are shown in a Reader Questions area below the public profile card, subject to the reader's public/private visibility choices.
- Removed the "Show on profile card" toggle from the admin question editor; legacy database fields are retained for backwards compatibility.